PS 2010: Intro to R

Qing Chang

09/01/2022

Topic 1: Useful R commands

The following commands are relevant for future tutorials.

Commands to create sequences and vectors:

```
1:10 # Creates a sequence of integers
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

```
seq(1,10) # Does the same thing
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

```
seq(2,50,by=2) # Generate a sequence of number by increase the same amount
```

```
##  [1]  2  4  6  8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50
```

```
seq(1, 10, length.out = 5) # Generate a sequence by desired length
```

```
## [1]  1.00  3.25  5.50  7.75 10.00
```

```
newvector<-c(1,23,7,10,11,5,45) # Creates a vector with 7 elements
newvector
```

```
## [1]  1 23  7 10 11  5 45
```

```
rep(1,500) # Creates a vector with the number 1 repeated 500 times
```

```
##   [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##  [38] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##  [75] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [112] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [149] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [186] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [223] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [260] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [297] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [334] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [371] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [408] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [445] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [482] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
fun<-c("R","is","fun") # Creates a vector with non-numerical entries
fun
```

```
## [1] "R"   "is"  "fun"
```

```r
paste(fun, collapse=" ") # Turns the above vector into a single element
```

```
## [1] "R is fun"
```

Important basic built-in functions:

```r
abs(-10) # Returns the absolute value of a number
```

```
## [1] 10
```

```r
sqrt(4) # Gives you the square root of a number
```

```
## [1] 2
```

```r
log(exp(1)) # Gives you the natural logarithm, here ln(e) was entered e = exp(1)
```

```
## [1] 1
```

```r
log(100, base=10) # Gives you the logarithm of 100 with base 10
```

```
## [1] 2
```

```r
round(1.5) # Rounds the value to the next integer
```

```
## [1] 2
```

```r
sum(newvector) # Gives you the sum of all elements of a vector
```

```
## [1] 102
```

```r
length(newvector) # Gives you the length (number of elements) of a vector
```

```
## [1] 7
```

```r
mean(newvector) # Gives you the mean or average of a vector
```

```
## [1] 14.57143
```

```r
sum(newvector)/length(newvector) # Does the same thing (formula behind the above command)
```

```
## [1] 14.57143
```

```r
min(newvector) # Gives the minimum in a vector
```

```
## [1] 1
```

```r
max(newvector) # Gives the maximum in a vector
```

```
## [1] 45
```

```r
summary(newvector) # Gives you more detailed summary statistics
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00    6.00   10.00   14.57   17.00   45.00
```

```r
sort(newvector) # Reorder the values in the vector
```

```
## [1]  1  5  7 10 11 23 45
```

```r
sort(newvector, decreasing = T) # Reorder the values by descending
```

```
## [1] 45 23 11 10  7  5  1
```

```r
which(newvector >= 3) # Tells you which entries of the vector meet the condition of being larger than o
```

```
## [1] 2 3 4 5 6 7
```

If you have a logical vector, it can have two types of entries, TRUE (T) and FALSE (F).

TRUE and FALSE are possible values when conditions are formulated, such as "if" conditions within functions or loops.

```
logical <- c(T,F,T,F,F) # TRUE=T, FALSE=F
!logical # Will reverse the vector
```

```
## [1] FALSE  TRUE FALSE  TRUE  TRUE
```

```
3==4 # Double equal sign means testing whether two things are the same
```

```
## [1] FALSE
```

```
"This" == "That"
```

```
## [1] FALSE
```

```
3 < 4 # Less than
```

```
## [1] TRUE
```

```
3 <= 4 # Less than or equal to
```

```
## [1] TRUE
```

```
3 > 4 # Greater than
```

```
## [1] FALSE
```

```
3 >= 4 # Greater than or equal to
```

```
## [1] FALSE
```

```
3 != 4 # Not equal to
```

```
## [1] TRUE
```

```
x <- c(1,2,3)

2 %in% x # Testing whether a value is contained within a set
```

```
## [1] TRUE
```

If you have a vector, you can exclude elements from it.

```
a <- c(1,3,5,7,9)

b <- a[-3] # This command excludes the third element from the vector

b # Only 1, 2, 4, and 5 are left
```

```
## [1] 1 3 7 9
```

```
c <- a[-which(a==3)] # This command excludes the element that equals to 3.

c
```

```
## [1] 1 5 7 9
```

You can also exclude multiple elements from a vector with the following command.

```r
d <- a[-c(4,5)] # This command excludes the fourth and fifth element

d # Only 1, 2, and 3 are left
```

```
## [1] 1 3 5
```

Important: this command does not exclude values, it excludes items by their position!

```r
e <- c(11,15,9,2,34)

e <- d[-c(1,2)]

e # Only the last three items are left: 9, 2, and 34
```

```
## [1] 5
```

You can access individual elements of a vector with the following commands:

```r
a[3] # Gives you the third element of the vector
```

```
## [1] 5
```

```r
logical[2] # Gives you the second element of the vector
```

```
## [1] FALSE
```

You can also combine two vectors to a matrix.

```r
matrix=rbind(a,logical)
# Rbind stands for "rowbind", you bind two rows together,
#the alternative is "cbind", meaning that you bind columns together

matrix # The logical vector is turned numeric (1 for TRUE, 0 for FALSE)
```

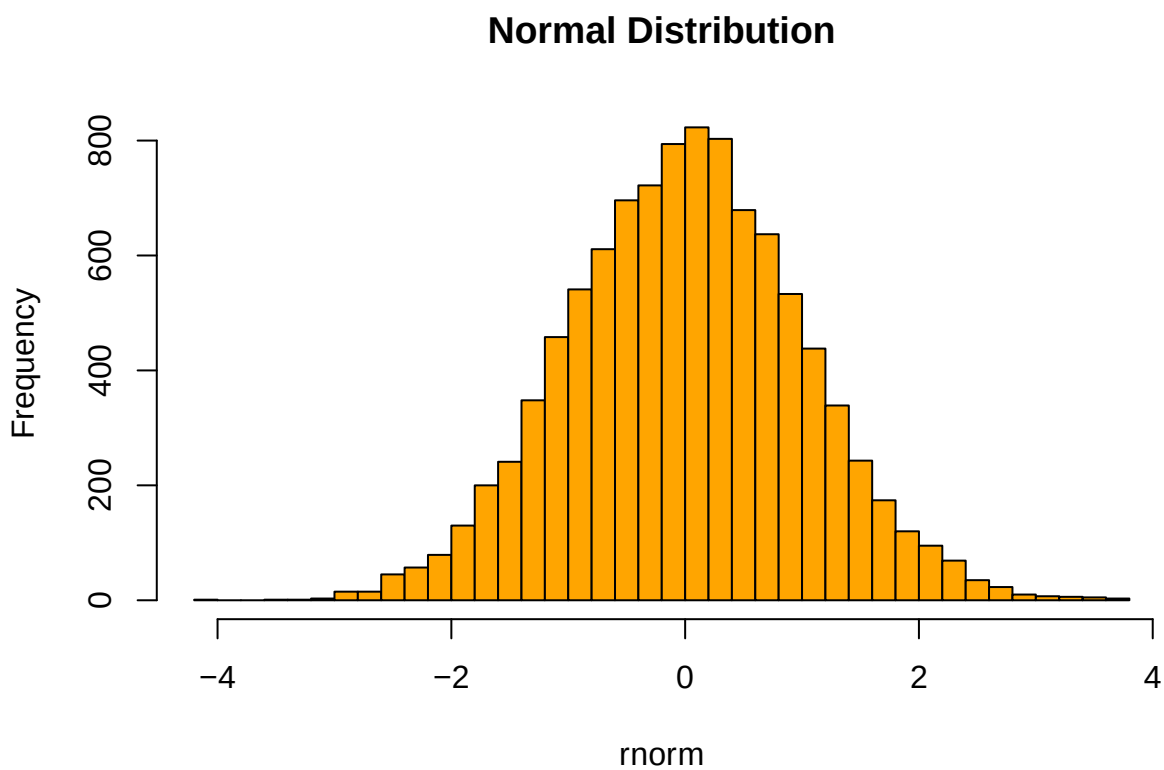
```
##         [,1] [,2] [,3] [,4] [,5]
## a          1    3    5    7    9
## logical    1    0    1    0    0
```

```r
matrix[2,3] # Gives you the element in the second row and third column
```

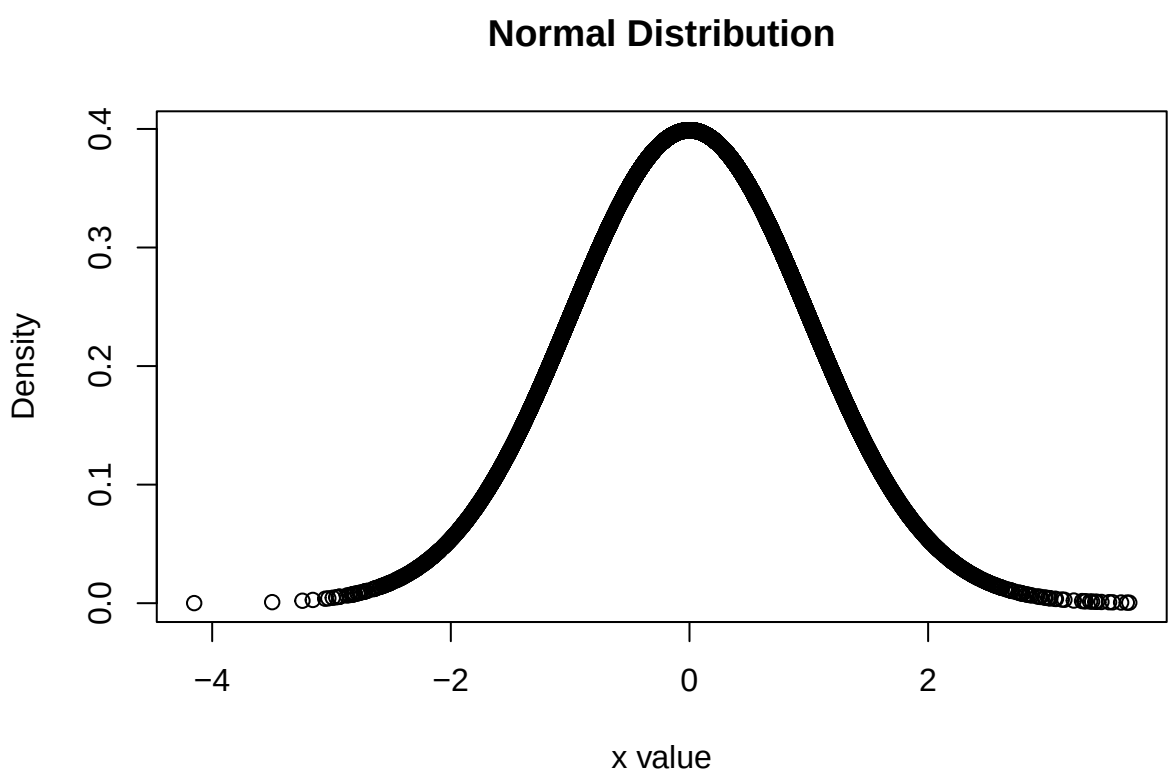
```
## logical
##       1
```

Draw Plot

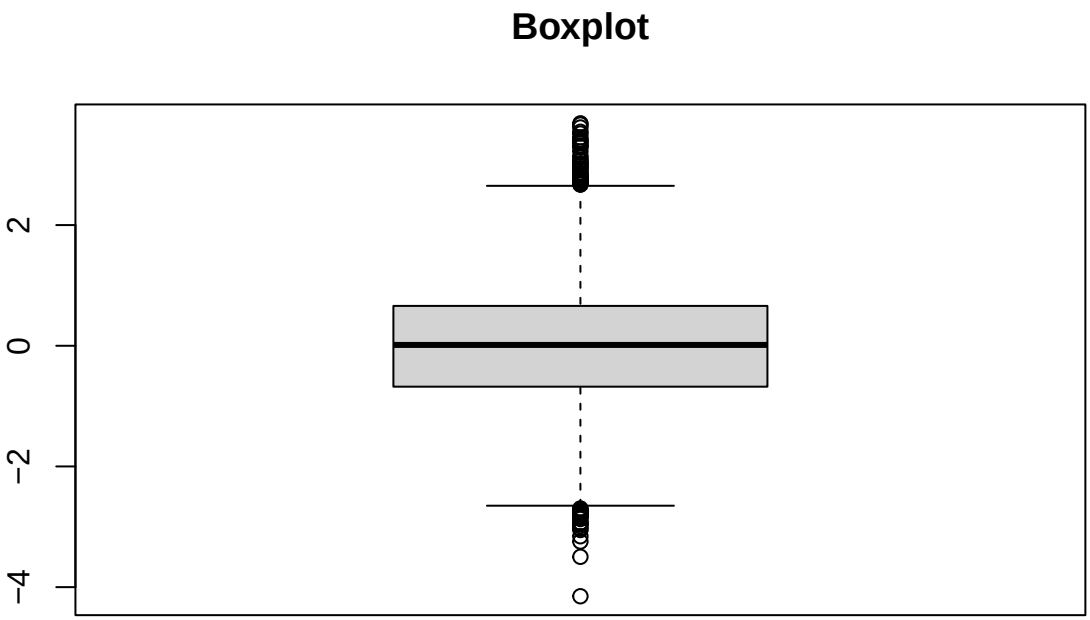
```r
rnorm <- rnorm(10000,0,1) #Random draw 10000 data from a standard normal distribtuon. The first argumen

hist(rnorm, breaks=30, col = "orange", main = "Normal Distribution")
```




```
plot(x=rnorm, y=dnorm(rnorm), xlab="x value", ylab="Density", main="Normal Distribution")
```

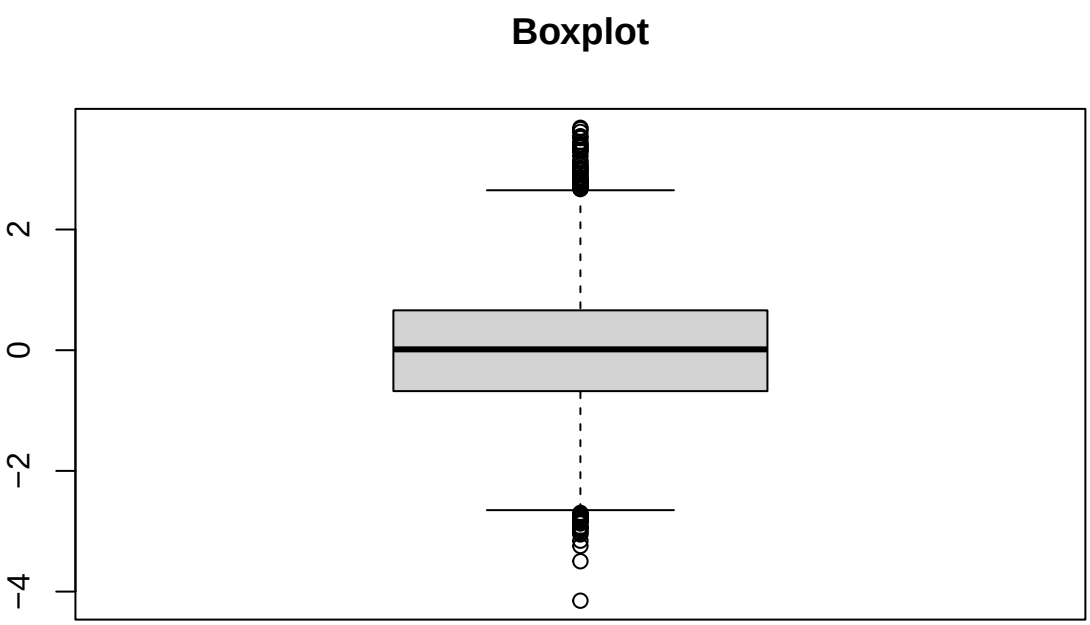



```
boxplot(rnorm, main="Boxplot")
```

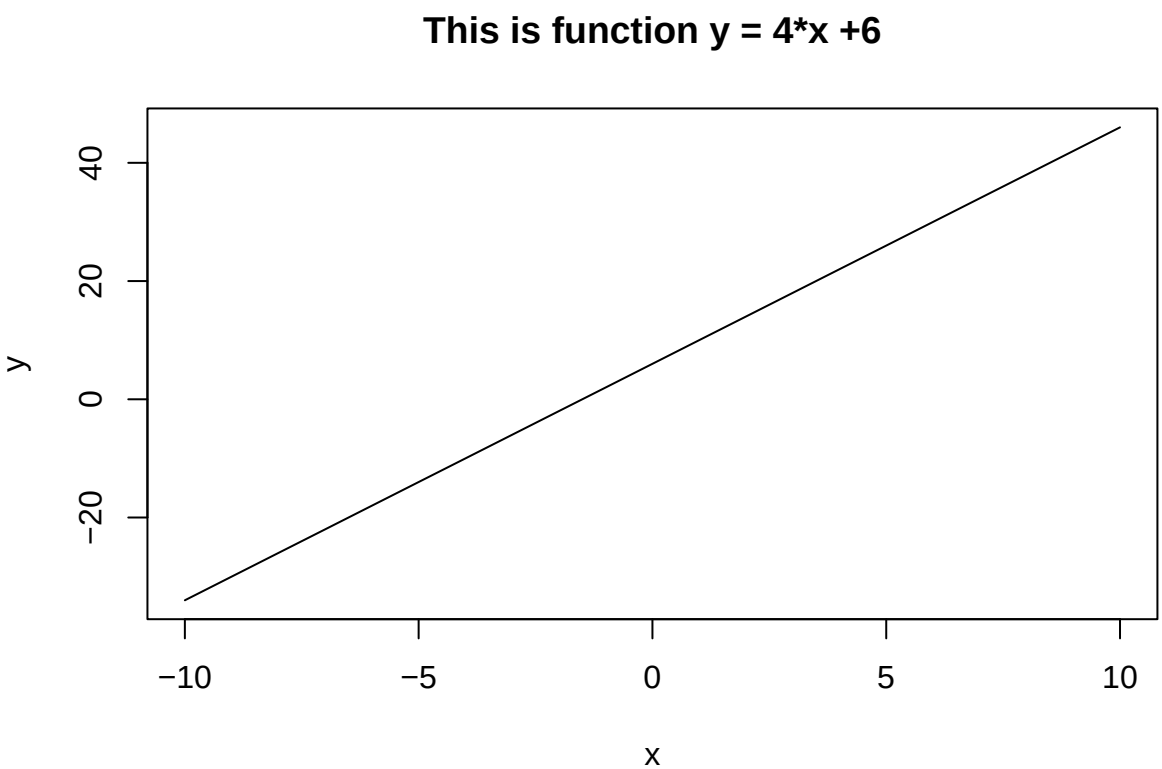


Plot Functions

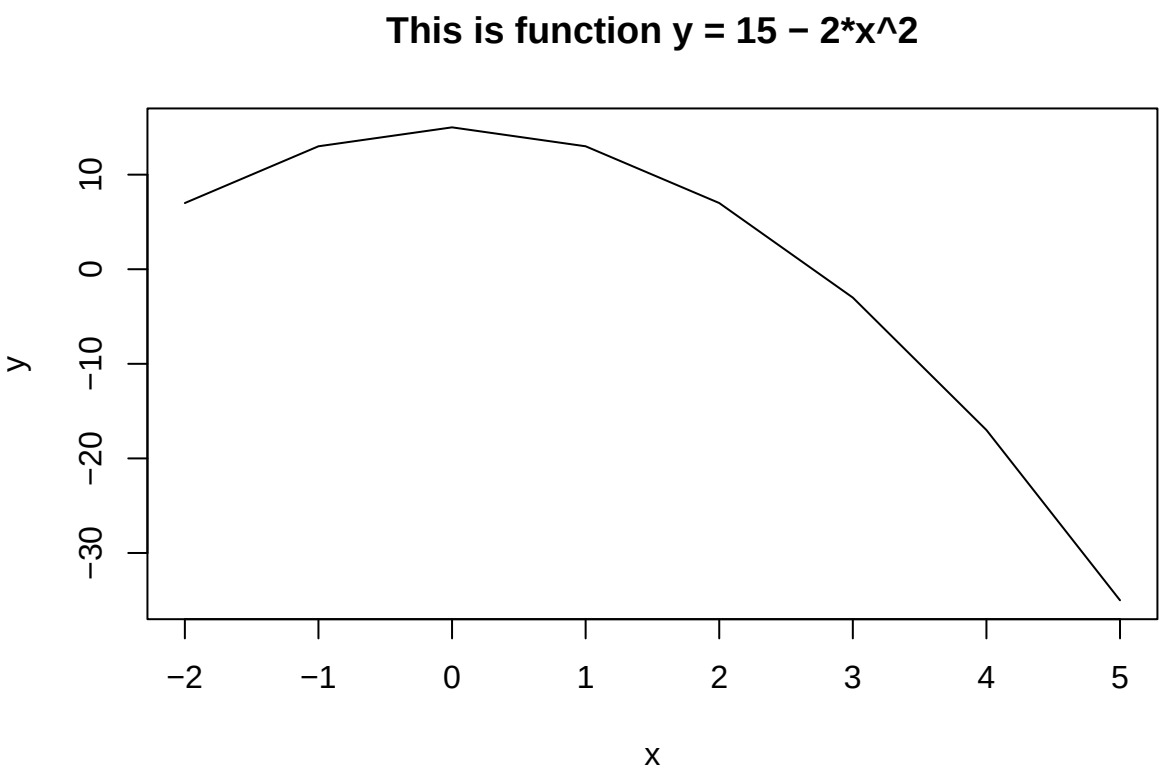
```
# A linear function: y = 4*x +6

x <- c(-10:10) # first, we set the domain
```



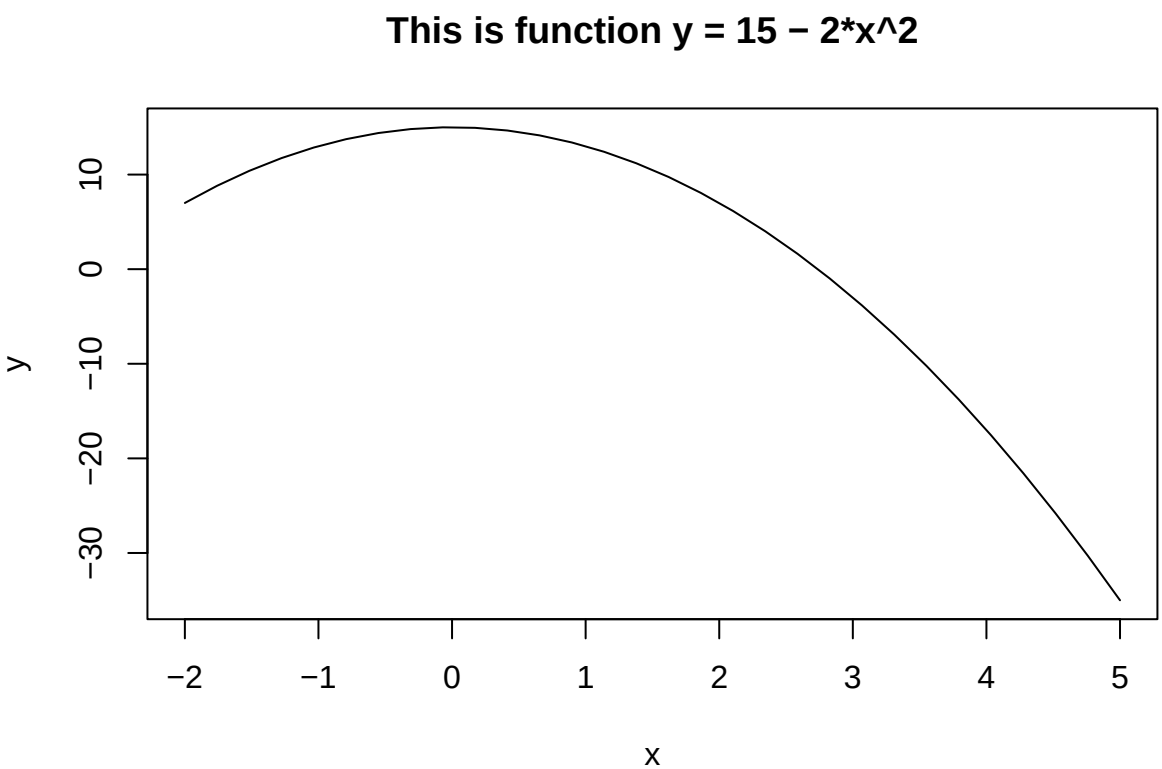

```
y <- 4*x +6 # then, we calculate the range


plot(x, y, type = "l", main = "This is function y = 4*x +6")
```



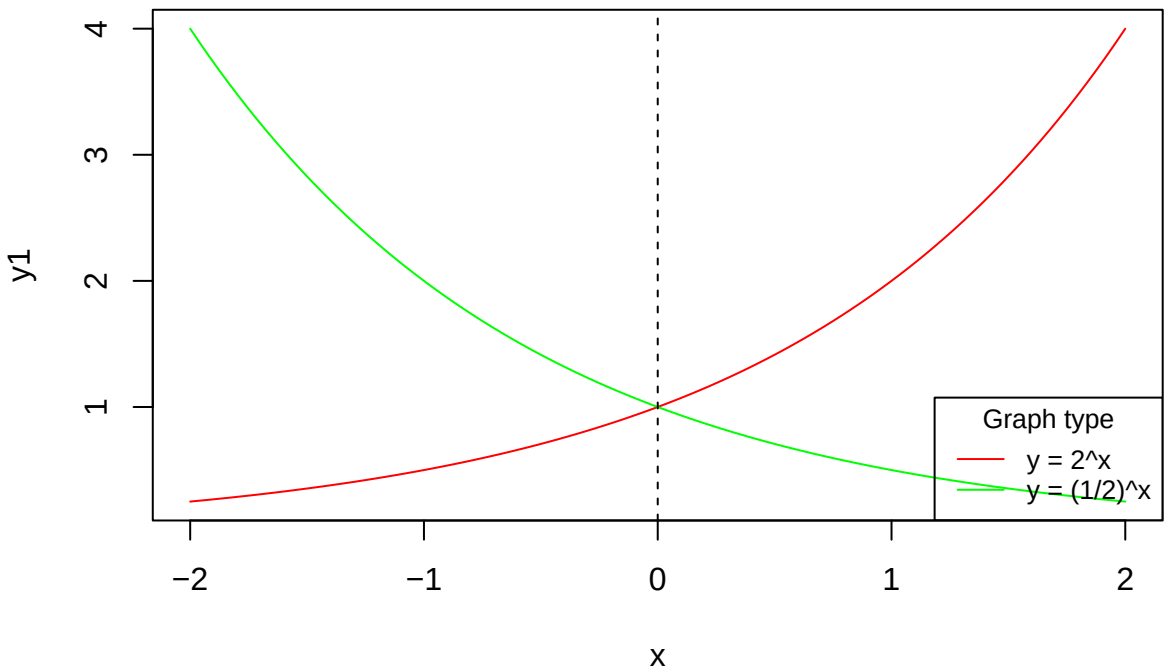

```r
# A non-linear function: y = 15 - 2*x^2

x <- c(-2:5)

y <- 15 - 2*x^2


plot(x, y, type = "l", main = "This is function y = 15 - 2*x^2")
```



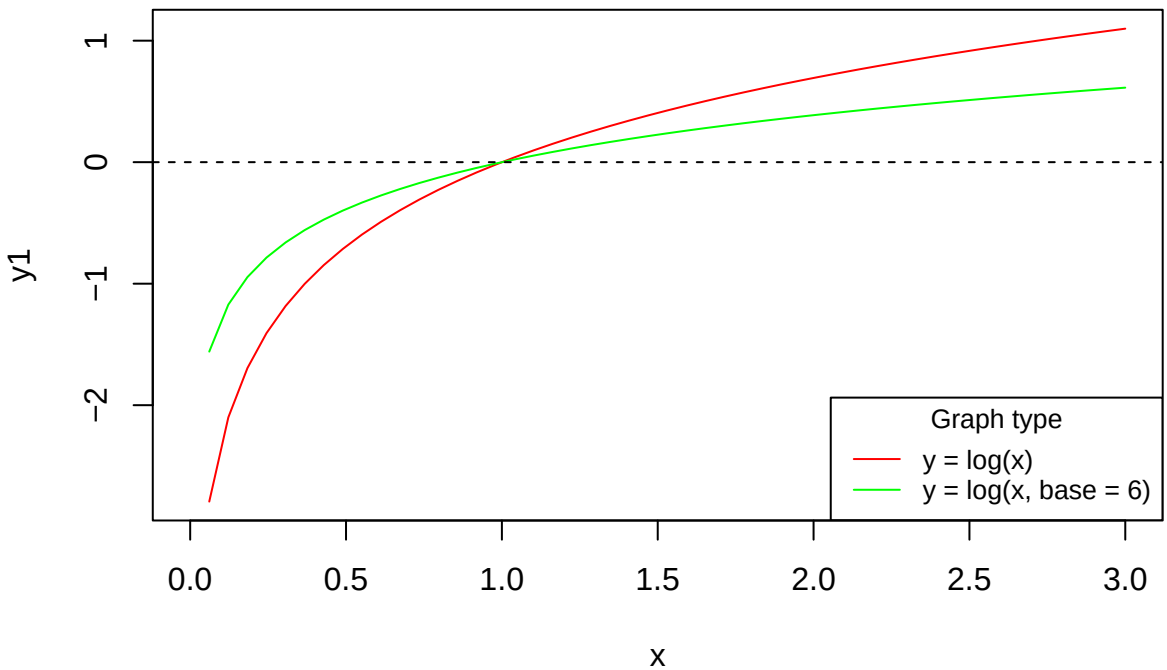

```r
## let's improve the plot by increase the number of values in domain

x <- seq(-2, 5, length.out = 30)

y <- 15 - 2*x^2


plot(x, y, type = "l", main = "This is function y = 15 - 2*x^2")
```



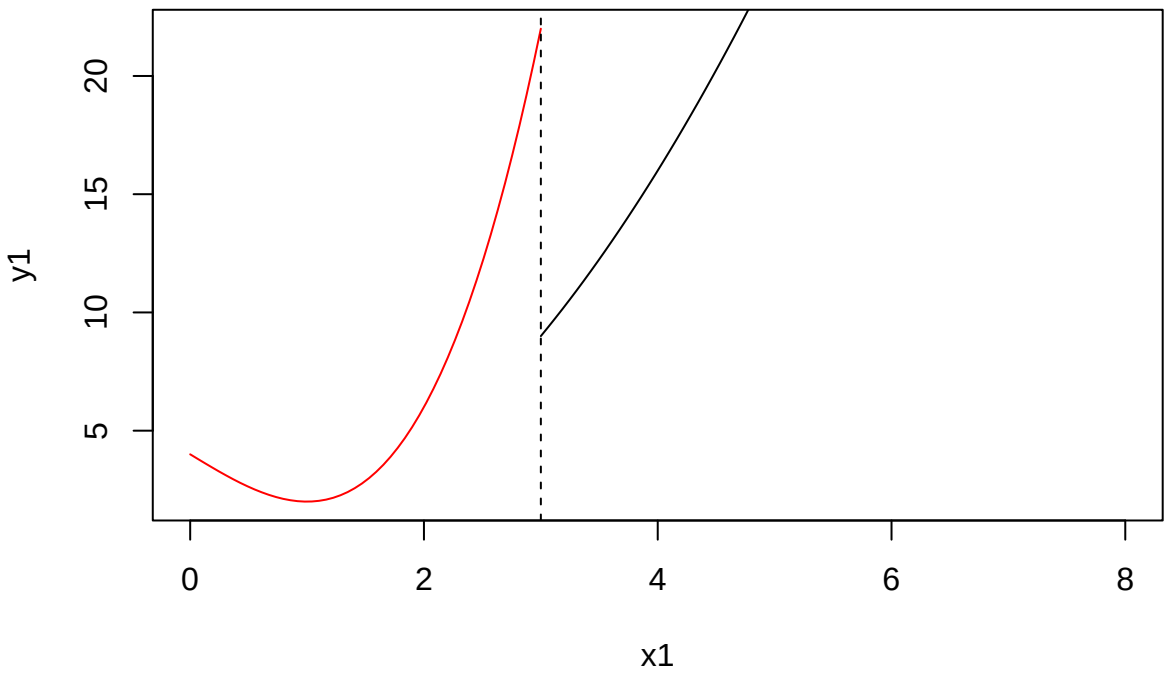

```r
# A exponential function: y = 2^x, and y = (1/2)^x

x <- seq(-2, 2, length.out = 100)

y1 <- 2^x

y2 <- (1/2)^x


{ # notice: we do not need this curly brackets in R script, but we need it here.
plot(x, y1, type = "l", col= "red")
lines(x, y2, col="green")
abline(v=c(0),lty=2)
legend('bottomright',legend = c("y = 2^x","y = (1/2)^x"),
       lty=1,col=c("red","green"),title="Graph type", cex=0.8)
}
```




```
# A logarithm function y = log(x), and y = log(x, base = 6)


x <- seq(0, 3, length.out = 50)

y1 <- log(x)

y2 <- log(x, base = 6)


{ # notice: we do not need this curly brackets in R script, but we need it here.
plot(x, y1, type = "l", col= "red")
lines(x, y2, col="green")
abline(h=c(0),lty=2)
legend('bottomright',legend = c("y = log(x)","y = log(x, base = 6)"),
       lty=1,col=c("red","green"),title="Graph type", cex=0.8)
}
```




```
# A discontinuous function:

x1 <- seq(0, 3, length.out = 50)

x2 <- seq(3, 6, length.out = 50)

y1 <- x1^3 - 3*x1 + 4

y2 <- x2^2

{ # notice: we do not need this curly brackets in R script, but we need it here.
plot(x1, y1, xlim = c(0,8), type = "l", col= "red")
lines(x2, y2, col="black")
abline(v=3, lty=2)
}
```


y1
20
15
10
5
0
2
4
6
8
x1